

Transformers with a Physics Payoff

Lecture 10 — Architecture + many-body kernel + evaluation

Prof. Young Woo Choi

Department of Physics, Sogang University

Week 8 · Session 1: transformer architecture & physics payoff · Session 2: evaluation/data pipeline

6/5/2026

Today's roadmap

Session 1: Transformer architecture

1. The whole picture — a stateless function
2. Self-attention: Q, K, V
3. Multi-head attention
4. Residual + RMSNorm
5. **Physics payoff** — attention as a data-dependent many-body kernel
6. GPT-2 family and scaling laws

Session 2: Evaluation & data pipeline

- Train / val / test split design
- Learning curves and overfitting signals
- Data leakage and how to detect it
- Cross-validation in small-data physics
- Term project model-comparison checklist

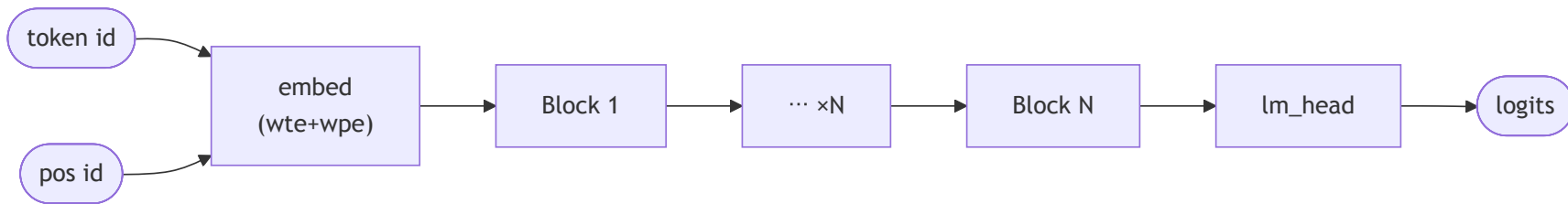
Data label: mixed — hands-on uses synthetic character sequences; physics payoff uses toy spin chains.

The whole picture

Input: one token id t , one position id p · **Output:** logits over the vocabulary — "what comes next?"

The model is a **stateless function** (params + KV cache in, logits out):

```
gpt(token_id, pos_id, keys, values) -> logits
```



Everything else is plumbing. Today we build this from scratch in **4 substantive steps**.

Step 1 — MLP baseline

Before any attention, a simple MLP over the token embedding predicts the next character.

```
# embed → hidden → logits
x = wte[token_id] + wpe[pos_id]    # (n_embd,)
h = relu(x @ W1 + b1)             # (n_hidden,)
logits = h @ W2 + b2              # (vocab_size,)
```

What's missing: each position is processed *independently* — no communication between tokens.

checkpoint	target
parameter count	~700
loss after 1000 steps	< 3.0
improvement over bigram baseline	yes

This is our **minimum viable** language model. Its failure to use context motivates attention.

Self-attention: Q, K, V

For each token, learn three projections:

Query **q**

"What am I **looking for**?"

$$\mathbf{q} = W_Q \mathbf{x}$$

Key **k**

"What do I **contain**?"

$$\mathbf{k} = W_K \mathbf{x}$$

Value **v**

"What do I **offer** if matched?"

$$\mathbf{v} = W_V \mathbf{x}$$

Then:

$$a_{ij} = \text{softmax}_j \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right) \quad \mathbf{x}_i^{\text{new}} = \sum_{j \leq i} a_{ij} \mathbf{v}_j$$

where $d = n_{\text{embd}}$ for **single-head** (Step 3). In Step 4 with multi-head, $d = d_{\text{head}} = n_{\text{embd}}/n_{\text{head}}$.

Step 2 — Autograd and the training loop

Before adding attention we need to train with gradients.

```
# manual gradient step
loss = cross_entropy(logits, target)
grads = grad(loss, params)          # step2: autograd
params = params - lr * grads
```

Checkpoint targets:

quantity	expected
gradient vs finite-difference	agreement < 1e-4
loss at step 0	≈ 3.6 (random init, vocab=27)
loss at step 1000	≈ 2.5

The loss at step 0 is $\ln(\text{vocab_size}) \approx \ln 27 \approx 3.3$ — sanity check your init.

Step 3 — Single-head attention

The attention sub-layer is the **only place** tokens communicate.

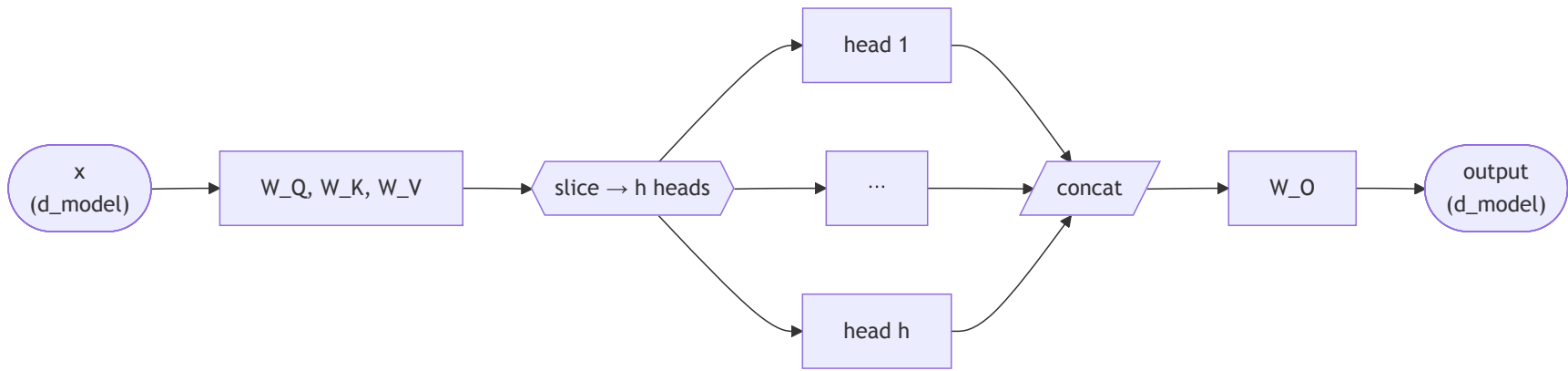
```
x_res = x
x = rmsnorm(x)
q = linear(x, Wq); k = linear(x, Wk); v = linear(x, Wv)
keys[1].append(k); values[1].append(v)          # KV cache
scores = [dot(q, kj) / sqrt(d) for kj in keys[1]] # d = n_embd here
scores = causal_mask_and_softmax(scores)
x_attn = sum(a * vj for a, vj in zip(scores, values[1]))
x = linear(x_attn, Wo)
x = [a + b for a, b in zip(x, x_res)]           # residual
```

Checkpoint: loss ≈ 2.3 (improvement over Step 2).

Scale note: the scores are normalized by $\sqrt{d}$, where $d = n_embd$ here for single-head. In Step 4 (multi-head) d becomes $d_{head} = n_embd / n_head$ — the scale changes per head. Implementing Step 3 with `n_embd` and copy-pasting into Step 4 without updating the scale is a common bug.

Step 4 — Multi-head attention + Adam

One head sees one kind of relationship. Slice Q, K, V into h independent heads.



$d_{\text{model}} = h \cdot d_{\text{head}}$ · same parameter budget, several attention patterns in parallel.

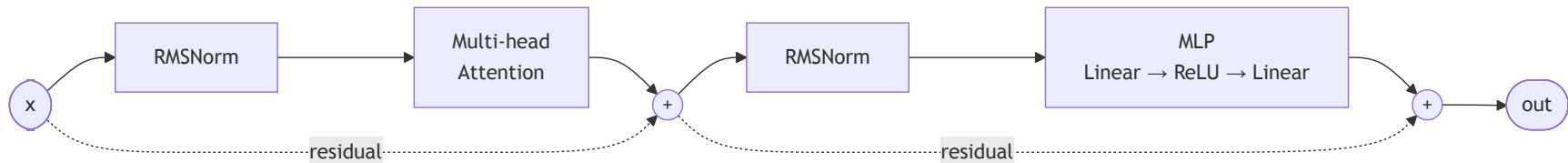
Adam is added in this step — momentum + adaptive learning rate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \theta_t = \theta_{t-1} - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Checkpoint: $n_{\text{layer}} = 1$ loss ≈ 2.3 ; $n_{\text{layer}} = 2$ loss ≈ 2.1 – 2.2 .

The Block: attention + MLP

Two sub-layers, same recipe: $y = x + \text{SubLayer}(\text{Norm}(x))$ (pre-norm).



Sub-layer	Role	Talks across tokens?
Attention	mix information across positions	yes
MLP	per-token feature transformation	no

Transformer = **Communication** (attention) interspersed with **Computation** (MLP).

Residual + RMSNorm

Residual: the "highway"

$$\mathbf{x}_{\ell+1} = \mathbf{x}_{\ell} + f(\mathbf{x}_{\ell})$$

- Gradients flow **directly** through the `+`
- Each block only learns a *correction*
- Physics analogy: perturbative correction to a free propagator

Without residuals, gradients vanish exponentially with depth.

RMSNorm: keep activations sane

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \varepsilon}}$$

- Rescales each token to **unit RMS**
- Simpler than LayerNorm (no mean subtraction)

```
def rmsnorm(x):
    ms = sum(xi*xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

Together: residuals make depth *trainable*, normalisation makes depth *stable*.

Physics payoff: attention as a many-body kernel

Many-body physics

Particle i "feels" all others through a kernel:

$$\phi_i^{\text{new}} = \sum_j K(i, j) \phi_j$$

Classical MLIP (SchNet): kernel is a *fixed* radial function of distance — neighbors within cutoff r_{cut} .

Self-attention

$$A_{ij} = \text{softmax}_j \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}} \right)$$

$$\mathbf{x}_i^{\text{new}} = \sum_{j \leq i} A_{ij} \mathbf{v}_j$$

The kernel A_{ij} is **data-dependent** — computed on-the-fly from the inputs themselves.

Key contrast: SchNet uses a **static neighborhood** (radial cutoff). GPT uses a **dynamic neighborhood** — A_{ij} is recomputed for every new input, selecting different "interacting

Physics payoff: toy spin chain demo

Apply the completed GPT to an $N = 8$ Ising spin chain: predict the next spin state s_{t+1} given the sequence $s_0, \dots, s_t$.

What to visualize:

```
# after training, extract attention weights at position t
A = attention_scores[t, :] # shape (t+1,)
# plot A as a bar chart over past positions
```

SchNet (static)

Neighbors: all j with $|r_i - r_j| < r_{\text{cut}}$

Fixed for every input — order-independent, geometry-only.

GPT (dynamic)

Neighbors: weighted by A_{ij} — different for each context.

A_{ij} can be large for distant j if the context makes it relevant.

Forward pointer: MBFormer and set-transformer (L20)

The data-dependent kernel framing connects directly to frontier physics applications.

Set-transformer on point clouds (L11)

Replace the causal sequence with an **unordered set** of atoms or particles.

- Remove positional embedding (order-agnostic)
- Replace causal mask with full attention (no past/future)
- Output: permutation-invariant property

MBFormer (L20 survey)

Many-body transformer for quantum chemistry: attention heads learn which orbital pairs matter for the many-body wavefunction.

$A_{ij} \leftrightarrow$ effective coupling between orbital i and orbital j — a **learned** Slater determinant.

GPT-2 family and scaling laws

Model	Layers N	d_{model}	Heads	Parameters
GPT-2 small	12	768	12	117 M
GPT-2 medium	24	1024	16	345 M
GPT-2 large	36	1280	20	762 M
GPT-2 XL	48	1600	25	~1.5 B (1542 M)
GPT-3	96	12288	96	175 B

Error fix #15: GPT-2 XL had ~1.5 billion parameters (1542 M), not "1.6 billion". The largest GPT-2 variant is XL; GPT-3 is a separate, much larger model.

Map back to the microgpt code

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id]           # ← embedding
    pos_emb = state_dict['wpe'][pos_id]             # ← positional
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x)

    for li in range(n_layer):                       # ← stack N blocks
        # — Attention sub-layer —————
        x_residual = x
        x = rmsnorm(x)                               # ← pre-norm
        q = linear(x, ...wq); k = linear(x, ...wk); v = linear(x, ...wv)
        keys[li].append(k); values[li].append(v)     # ← KV cache
        for h in range(n_head):                     # ← multi-head
            # scale by  $\sqrt{d_{\text{head}}} = \sqrt{(n_{\text{embd}}/n_{\text{head}})}$ , NOT  $\sqrt{n_{\text{embd}}}$ 
            ... attn_logits /  $\sqrt{d_{\text{head}}}$  → softmax → weighted v
        x = linear(x_attn, ...wo)
        x = [a + b for a, b in zip(x, x_residual)]    # ← residual

        # — MLP sub-layer —————
        x_residual = x
        x = rmsnorm(x)                               # ← pre-norm
        x = linear(x, ...fc1); x = [xi.relu() for xi in x]; x = linear(x, ...fc2)
        x = [a + b for a, b in zip(x, x_residual)]    # ← residual
```

Recap — four architecture ideas

1. Self-attention

Data-dependent interaction kernel. Q asks, K answers, V is what flows. Scale by $\sqrt{d_{\text{head}}}$ in multi-head. Causal mask: only past $\rightarrow$ present.

2. Multi-head

Same parameter budget, several patterns in parallel. Slice d_{model} into h chunks of d_{head} . Scale: $\sqrt{d_{\text{head}}}$, not $\sqrt{d_{\text{model}}}$.

3. Residual + RMSNorm

Highway for gradients + activation rescaling. Makes deep stacks both **trainable** and **stable**.

4. Block stack

Communication (attn) $\leftrightarrow$ Computation (MLP), repeated N times on a shared residual stream.

Session 2: Evaluation, validation, and data pipelines

This session is a practical prerequisite for the **term project** (50% of grade).

Why a dedicated segment? Evaluation mistakes are the single most common source of inflated results in ML-for-physics papers:

- Training on the test set (data leakage)
- Reporting best validation metric without a held-out test set
- Comparing models with different preprocessing
- Underfitting masked by insufficient training epochs

Checkpoint for this session: by the end, each student will have a draft of their term project's (1) split design, (2) validation loss curve plan, and (3) model comparison table structure.

Train / val / test split

The three roles

Split	Purpose	Touches model?
Train	gradient updates	yes
Val	hyperparameter tuning, early stopping	indirectly
Test	final unbiased evaluation	once, at the end

Rule: the test set is a sealed envelope. Opening it more than once invalidates the estimate.

Typical sizes

Dataset size	Train / Val / Test
large ($\$ > \$10k$)	80 / 10 / 10 %
medium (1k–10k)	70 / 15 / 15 %
small ($\$ < \$1k$)	cross-validation + held-out test

Physics note: for MD trajectories, split by **time** (or by structure/molecule), not randomly — adjacent frames are correlated.

Learning curves

A learning curve plots loss (or error) vs. training set size *or* training step.

Overfitting signal

Train loss keeps decreasing; val loss plateaus or rises.

Fix: more data, stronger regularization, early stopping, dropout.

Underfitting signal

Both train and val loss are high and plateau together.

Fix: larger model, more capacity, better features, longer training.

Physics analogy: train loss $\approx$ in-sample energy; val loss $\approx$ free energy on new configurations. Overfitting is like a model that perfectly memorizes training configurations but cannot extrapolate.

Data leakage: the silent killer

Data leakage = information from the test set (or future) flows into training.

Common sources

1. **Preprocessing fitted on all data** (e.g., mean/std computed on full dataset, then split)
2. **Temporal leakage** — future frames in training set
3. **Duplicate or near-duplicate samples** across splits
4. **Target leakage** — features that encode the label (e.g., include energy in features when predicting energy)

How to detect it

- Val performance unrealistically close to train performance
- Model generalizes perfectly to val but fails on new data
- Feature importance analysis reveals "too-informative" features

Correct pipeline:

```
# WRONG: fit scaler on all data
scaler.fit(X_all); X_all_scaled = scaler.transform(X_all)

# CORRECT: fit scaler only on training data
scaler.fit(X_train); X_train_s = scaler.transform(X_train)
X_val_s = scaler.transform(X_val) # transform, do not fit
```

Cross-validation in small-data physics

When data is scarce (common in DFT/experiment), k -fold cross-validation gives a reliable estimate.

Model comparison checklist

When comparing two models (baseline vs proposed), the comparison is only valid if:

Same data

- Same train/val/test split
- Same preprocessing pipeline (fit on train only)
- Same random seed for split

Same evaluation

- Same metric (MAE, RMSE, cross-entropy, ...)
- Same test set, evaluated once
- Report confidence interval ($\pm$ std over seeds or folds)

Fair compute

- Hyperparameters tuned with equal effort for each model

Ablation

- At least one ablation: remove your key idea and measure drop
- Report both with and without

Term project data pipeline checkpoint

In-class exercise (15 min): answer these three questions for your term project.

1. Split design

- How large is your dataset?
- What are your train / val / test sizes and why?
- If time-series or correlated: how do you split?

2. Learning curve plan

- What metric(s) will you plot?
- How will you detect overfitting?
- What is your early-stopping criterion?

3. Model comparison table

- What is your baseline model?
- What is your proposed model?
- What is the primary metric for comparison?

Submit a one-paragraph data pipeline description with your project proposal. This is a graded component.

Optional: HuggingFace GPT-2 fine-tuning (appendix)

Bridge from "how it works" to "how it's used in practice":

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer, TrainingArguments

model = GPT2LMHeadModel.from_pretrained("gpt2") # 124 M, OpenAI pretrained
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

# Fine-tune on a small physics corpus (arXiv abstracts, ~10k sentences)
# Colab T4: ~10 minutes
training_args = TrainingArguments(output_dir="./gpt2-physics", num_train_epochs=3, ...)
trainer = Trainer(model=model, args=training_args, train_dataset=physics_dataset)
trainer.train()
```

What this demonstrates:

- The weights you just built from scratch are the *same architecture* as `gpt2`
- Fine-tuning reuses the pretrained representations and specializes the last layers
- This is the same pattern as ImageNet transfer in L07, or MACE-MP-0 fine-tuning in L13

Summary

Session 1: Architecture

- 4-step microgpt build: MLP → autograd → single-head → multi-head + Adam
- Attention = data-dependent many-body kernel
- Dynamic vs static neighborhoods (GPT vs SchNet)
- GPT-2 XL: ~1.5 B params (1542 M)
- Forward pointer: L11 GNN, L20 MBFormer/set-transformer

Session 2: Evaluation

- Train / val / test: three distinct roles
- Leakage: fit preprocessing only on training data
- Learning curves: gap = overfitting signal
- Cross-validation for small physics datasets
- Term project checkpoint: split design + metric + comparison table

Next: L11 — Graph Neural Networks & Geometric DL (QM9 real molecules). The permutation

Further reading

- **Karpathy**, *Let's build GPT from scratch* — github.com/karpathy/ng-video-lecture
- **Vaswani et al. (2017)**, *Attention Is All You Need* — arxiv.org/abs/1706.03762
- **Jay Alammar**, *The Illustrated Transformer* — jalammar.github.io/illustrated-transformer
- **3Blue1Brown**, *Attention in transformers* — 3blue1brown.com/lessons/attention
- **poloclub**, *Transformer Explainer* — poloclub.github.io/transformer-explainer (live GPT-2 in the browser)
- **Kaplan et al. (2020)**, *Scaling Laws for Neural Language Models* — arxiv.org/abs/2001.08361
- **Lee et al. (2019)**, *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks* — arxiv.org/abs/1810.00825
- **Hastie, Tibshirani, Friedman**, *The Elements of Statistical Learning* Ch. 7 — model assessment and selection